

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Модели решения задач
в интеллектуальных системах»
на тему:

**Реализация и анализ алгоритмов обучения многослойных
нейронных сетей. Применение нейронных сетей
для сжатия и восстановления образов.**

Студент гр. 321701
Руководитель

И. А. Политыко
В. П. Ивашенко

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений	3
1 Постановка задачи	4
2 Реализация и результаты	5
Заключение	9
Список использованных источников	10

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

BMP	— Bitmap, Растровый формат изображения;
НС	— Нейронная сеть;
СКО	— Среднеквадратичная ошибка;
X	— Эталонный вектор блока изображения;
Y	— Сжатое представление блока;
X'	— Восстановленный вектор блока;
W	— Матрица весов прямого распространения;
W'	— Матрица весов обратного распространения;
n	— Размерность входного (и выходного) вектора;
p	— Число нейронов скрытого слоя;
Q	— Число блоков изображения;
Z	— Коэффициент сжатия;
α	— Коэффициент обучения.

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы является ознакомление, анализ и приобретение навыков реализации модели линейной рециркуляционной сети для задачи сжатия графической (или иной) информации.

Вариант задания — адаптивный коэффициент обучения и нормированные веса (`ADAPTIVE = True`, `NORMALIZE_WEIGHTS = True`).

Требуется:

- а) построить модель линейной рециркуляционной сети, решающую задачу сжатия графических данных;
- б) построить семейства графиков зависимостей параметров модели;
- в) предоставить письменный отчёт.

Исходные данные — растровое изображение формата BMP. Изображение размера $h \times w$ разбивается на Q прямоугольных блоков размера $r \times m$, где $4 \leq r \leq h$, $4 \leq m \leq w$. Значения интенсивности нормализуются в диапазон $[-1, 1]$ по формуле

$$c = \frac{2C}{C_{\max}} - 1, \quad C_{\max} = 255. \quad (1.1)$$

Каждый блок преобразуется в эталонный вектор $[X]$ длины $n = r \cdot m \cdot |S|$, где $|S|$ — число цветовых каналов.

2 РЕАЛИЗАЦИЯ И РЕЗУЛЬТАТЫ

Линейная рециркуляционная сеть выполняет автоассоциативное отображение вектора блока через узкий скрытый слой. Прямое распространение сжимает вектор, обратное — восстанавливает его:

$$Y = XW, \quad X' = YW', \quad (2.1)$$

где W имеет размер $n \times p$, W' — размер $p \times n$, $p < n$.

Схема преобразования показана на рисунке 2.1.

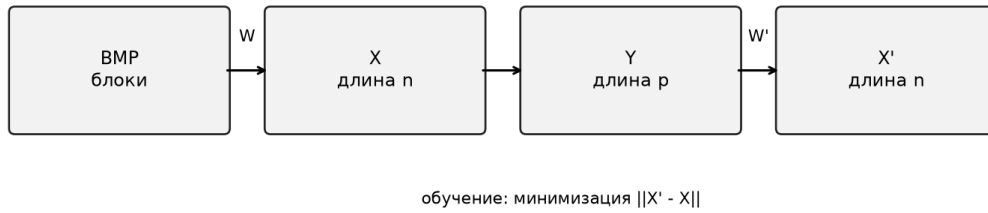


Рисунок 2.1 — Схема линейной рециркуляционной сети

Обучение выполняется по правилу коррекции ошибки. Для каждого эталона вычисляется $\varepsilon = X' - X$, после чего обновляются веса. В реализованном варианте коэффициент обучения адаптивный:

$$\alpha = \frac{1}{\|X\|^2 + \|Y\|^2}. \quad (2.2)$$

После обновления выполняется нормирование столбцов матрицы W и строк матрицы W' к единичной длине.

Суммарная ошибка эпохи

$$E = \frac{1}{2} \sum_{k=1}^Q \|X'_k - X_k\|^2. \quad (2.3)$$

Обучение продолжается до достижения порога $E \leq \varepsilon$ либо до исчерпания лимита эпох.

Коэффициент сжатия рассчитывается по формуле методических указаний:

$$Z(Q, n, p) = \frac{Q \cdot n}{(n + Q) \cdot p}. \quad (2.4)$$

Требуется $Z > 1$.

Программа реализована на языке Python с использованием библиотек NumPy, Pillow и Matplotlib. Структура модулей:

- `image_processing.py` — загрузка BMP, нарезка блоков, нормализация и денормализация, расчёт Z ;
- `network.py` — модель сети, обучение, сжатие и восстановление;
- `main.py` — интерактивный ввод параметров и сохранение результатов.

Ключевые фрагменты обучения приведены в листингах 2.1 и 2.2.

```

1 def _train_sample(self, X: np.ndarray) -> float:
2     Y = X @ self.W
3     Xp = Y @ self.Wp
4     err = Xp - X
5     alpha = self._alpha_for_sample(X, Y)
6     dWp = -alpha * np.outer(Y, err)
7     dW = -alpha * np.outer(X, err @ self.Wp.T)
8     self.Wp = self.Wp + dWp
9     self.W = self.W + dW
10    if NORMALIZE_WEIGHTS:
11        self._normalize_weights()
12    return float(np.dot(err, err))

```

Листинг 2.1 — Шаг обучения на одном блоке

```

1 def _alpha_for_sample(self, X: np.ndarray, Y: np.ndarray) -> float:
2     if not ADAPTIVE:
3         return self.alpha
4     denom = float(np.dot(X, X) + np.dot(Y, Y))
5     return 1.0 / max(denom, 1e-12)

```

Листинг 2.2 — Адаптивный коэффициент обучения

Эксперимент выполнен на тестовом цветном изображении 256×256 .
 Параметры: $r = m = 8$, $p = 32$, максимум 200 эпох, $\varepsilon = 0.01$. Получено $Q = 1024$ блоков, $n = 192$, коэффициент сжатия $Z \approx 5,05 > 1$.

Пример работы консольного интерфейса показан на рисунке 2.2.

```

Режим: adaptive, norm
Enter = по умолчанию

BMP [samples/test.bmp]:
r [8]:
m [8]:
p [32]:
эпохи [200]:
eps [0.01]:
shuffle (Y/n) [y]:
seed [42]:

img 256x256 c=3
8x8 Q=1024 n=192 p=32 Z=5.0526
режим: adaptive, norm
train...
epochs=200 E=150.409148 mse=29.5278
out: ...\\lab3\\output

```

Рисунок 2.2— Пример запуска программы

Исходный фрагмент изображения и результат восстановления приведены на рисунках 2.3 и 2.4. График суммарной ошибки по эпохам — на рисунке 2.5.

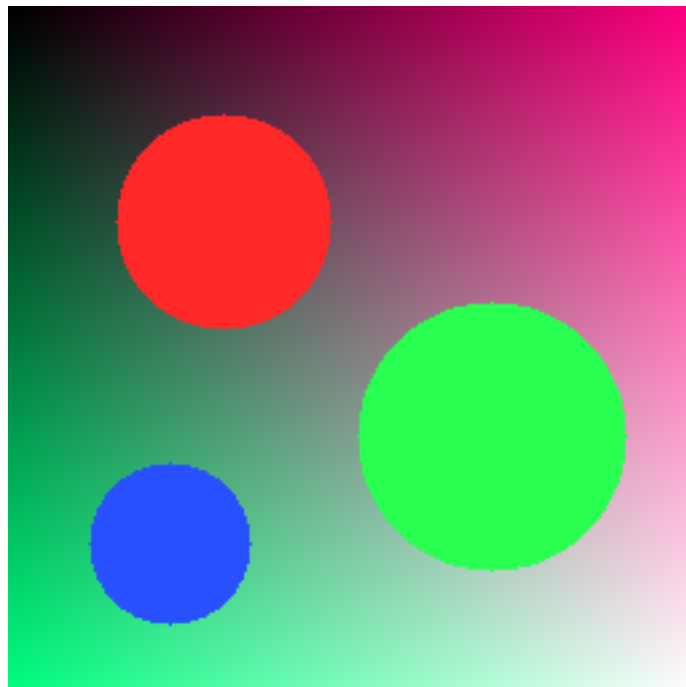


Рисунок 2.3— Исходное изображение (обрезанная область)

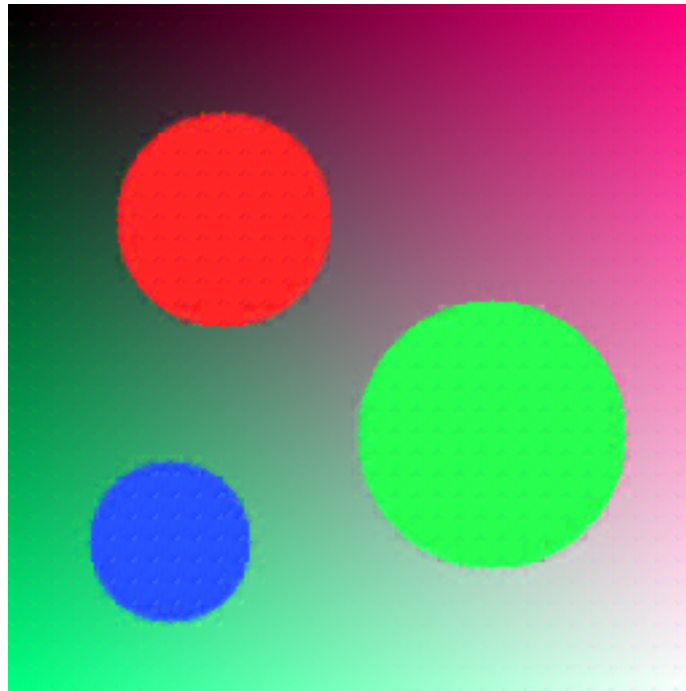


Рисунок 2.4— Восстановленное изображение

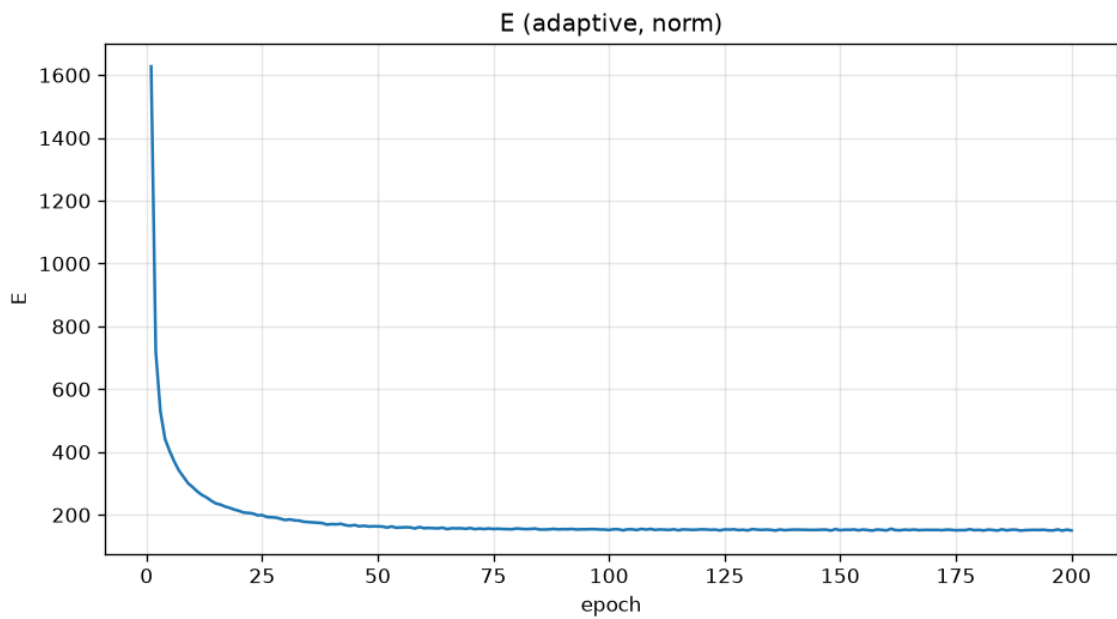


Рисунок 2.5— Зависимость суммарной ошибки E от номера эпохи

По результатам прогона: $E \approx 150,4$, средняя ошибка по пикселям $MSE \approx 29,5$. Ошибка быстро убывает в начале обучения и выходит на плато, что соответствует сходимости модели при выбранном p . Визуально восстановленное изображение сохраняет крупные цветовые области и контуры фигур при допустимых искажениях, обусловленных сжатием с потерями ($p < n$).

ЗАКЛЮЧЕНИЕ

В лабораторной работе реализована линейная рециркуляционная сеть для сжатия и восстановления растровых изображений ВМР. Изображение разбивается на блоки, нормализуется и обучается автоассоциативно через скрытый слой размерности p . Для варианта с адаптивным коэффициентом обучения и нормированными весами получено сжатие $Z > 1$, показаны исходное и восстановленное изображения, а также график убывания суммарной ошибки по эпохам.

Семантическая составляющая. Сжатие интерпретируется как отображение вектора блока в пространство меньшей размерности с последующим обратным отображением; качество определяется согласованностью матриц W и W' и выбором p .

Прагматическая составляющая. Получен воспроизводимый программный стенд на Python с интерактивным запуском, сохранением сжатых представлений, весов, метрик и графиков, пригодный для сравнения режимов обучения и параметров блока.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Методические указания к лабораторной работе №3 по дисциплине «Модели решения задач в интеллектуальных системах». — Минск : БГУИР.
- [2] Хайкин С. Нейронные сети: полный курс. — 2-е изд. — М. : Вильямс, 2006. — 1104 с.
- [3] NumPy documentation [Электронный ресурс]. — Режим доступа: <https://numpy.org/doc/>. — Дата доступа: 02.10.2026.
- [4] Pillow (PIL Fork) documentation [Электронный ресурс]. — Режим доступа: <https://pillow.readthedocs.io/>. — Дата доступа: 02.10.2026.